



UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

LUCRARE DE LICENȚĂ

MindBuddy: An AI-Based Mental Health
Companion for Emotional Support

COORDONATOR:

Lect. Univ. Dr.
Ștefăniță Sebastian-Aurelian

ABSOLVENT:

Tămășilă Vlad-Trandafir

TIMIȘOARA

2026



UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII DE LICENȚĂ: Informatică

LUCRARE DE LICENȚĂ

MindBuddy: An AI-Based Mental Health
Companion for Emotional Support

COORDONATOR:

Lect. Univ. Dr.
Ștefăniță Sebastian-Aurelian

ABSOLVENT:

Tămășilă Vlad-Trandafir

TIMIȘOARA

2026

Abstract

This thesis presents the design and implementation of *MindBuddy*, an AI-based emotional support companion built as a native Android application that communicates in Romanian. The system addresses the limited availability of mental-health support tools that work in Romanian and that give the user concrete control over the data that is stored.

The application is built on a three-tier client-server architecture: a native Android client written in Kotlin with Jetpack Compose, a Node.js/Express backend and a PostgreSQL database. Conversational responses and emotion classification are produced through the DeepSeek API.

The author personally designed and implemented the entire system. The main contributions are: a safety-check module that detects mentions of suicidal thoughts through both the language model and an auxiliary pattern-matching layer, progressively escalates support, and offers two recovery paths — the PHQ-2 screening test and validation by a previously designated trusted contact — with device-level persistence of the crisis state; a private mode in which the conversation content is not saved to the database, the context being kept only in memory; and a graphical visualization of emotional evolution over configurable time intervals. These build on a complete implementation of the supporting components: an authentication system with email verification, password reset and JWT-based sessions stored in encrypted form on the device; a local application lock based on a four-digit PIN derived with PBKDF2-SHA256, with optional biometric unlock; and a local reminder system implemented with WorkManager.

MindBuddy does not aim to replace specialized psychological intervention; it proposes a complementary digital tool, available at any time and oriented toward prevention and emotional self-awareness.

Keywords: mental health; AI companion; natural language processing; native Android application; emotional monitoring.

Rezumat

Lucrarea de față prezintă proiectarea și implementarea aplicației *MindBuddy*, un companion de suport emoțional bazat pe inteligență artificială, dezvoltat ca aplicație Android nativă care comunică în limba română. Sistemul răspunde lipsei unor instrumente de sprijin pentru sănătatea mintală disponibile în română și care să ofere utilizatorului control concret asupra datelor stocate.

Aplicația este construită pe o arhitectură client-server pe trei niveluri: un client Android nativ scris în Kotlin cu Jetpack Compose, un backend Node.js/Express și o bază de date PostgreSQL. Răspunsurile conversaționale și clasificarea emoțiilor sunt generate prin API-ul DeepSeek.

Întregul sistem a fost proiectat și implementat personal. Principalele contribuții sunt: un modul de safety check care detectează mențiunile de gânduri suicidale atât prin modelul de limbaj, cât și printr-un strat auxiliar de pattern matching, escalează progresiv sprijinul și oferă două căi de revenire — testul PHQ-2 și validarea de către un contact de încredere desemnat în prealabil — cu persistența stării de criză la nivel de dispozitiv; un mod privat în care conținutul conversațiilor nu este salvat în baza de date, contextul fiind păstrat exclusiv în memorie; și vizualizarea grafică a evoluției emoționale pe intervale de timp configurabile. Acestea se sprijină pe o implementare completă a componentelor de bază: un sistem de autentificare cu verificare prin email, resetare parolă și sesiuni JWT stocate criptat pe dispozitiv; o blocare locală prin PIN de patru cifre derivat cu PBKDF2-SHA256, cu deblocare biometrică opțională; și un sistem de reminders locale prin WorkManager.

MindBuddy nu urmărește înlocuirea intervențiilor psihologice specializate, ci propune un instrument digital complementar, accesibil oricând și orientat spre prevenție și auto-conștientizare emoțională.

Cuvinte cheie: sănătate mintală; companion AI; procesarea limbajului natural; aplicație Android nativă; monitorizare emoțională.

Declarație de integritate științifică

Declar că această lucrare de licență reprezintă un raport original și constituie propria mea muncă, realizată sub coordonarea Lect. Dr. Ștefanigă Sebastian-Aurelian. În această lucrare, metodele de cercetare, precum și procesele de colectare, analiză și interpretare a datelor, sunt prezentate corect, iar toate sursele au fost citate corespunzător acolo unde am utilizat idei și/sau materiale vizuale aparținând altor autori.

Am utilizat instrumente de inteligență artificială generativă în procesul de redactare și cercetare. Detaliile se regăsesc în Anexa A.

Cuprins

1	Introducere	8
2	Descrierea Problemei și Abordări Existente	10
2.1	Problema abordată	10
2.2	Procesarea limbajului natural și analiza sentimentelor	11
2.3	Abordări existente și comparație	11
3	Arhitectura Aplicației	13
3.1	Arhitectura generală	13
3.2	Componentele sistemului	14
3.2.1	Aplicația Android	14
3.2.2	Backend-ul Node.js	14
3.2.3	Baza de date PostgreSQL	15
3.2.4	Servicii externe	16
3.3	Diagrama cazurilor de utilizare	16
3.4	Diagrama de secvență — trimiterea unui mesaj	17
4	Funcționalitățile Aplicației	18
4.1	Autentificarea și gestionarea contului	18
4.1.1	Înregistrare și verificare prin email	18
4.1.2	Autentificarea și sesiunea JWT	18
4.1.3	Resetarea parolei	19
4.2	Conversațiile cu context persistent	19
4.2.1	Crearea și gestionarea conversațiilor	19
4.2.2	Contextul persistent: o strategie la nivel de aplicație	19
4.2.3	Istoricul conversațiilor	20
4.3	Modul privat	20
4.4	Modul de analiză a emoțiilor	21
4.5	Blocarea aplicației cu PIN și amprentă	21
4.6	Reminders și notificări locale	22
4.7	Modulul de safety check (verificare de siguranță)	23
4.7.1	Pragul „îngrijorător”: reminders forțate	23
4.7.2	Pragul „criză”: blocajul conversațiilor	23
4.7.3	Deblocarea în două căi	24

4.7.4	Contactul de încredere obligatoriu	24
4.7.5	Persistența la nivel de dispozitiv	25
4.7.6	Detecția crizei în modul privat	25
4.8	Interfața cu utilizatorul	25
5	Detalii de Implementare	26
5.1	Tehnologii și instrumente utilizate	26
5.2	Implementarea autentificării	26
5.2.1	Hashing-ul parolelor	26
5.2.2	Generarea și validarea JWT	27
5.3	Interceptorul de autentificare OkHttp	27
5.4	Clasificarea emoțiilor	27
5.5	Securitatea datelor	28
5.6	Implementarea modulelor de protecție și siguranță	28
5.6.1	PIN-ul local cu derivare PBKDF2	28
5.6.2	Detecția mențiunilor suicidale cu strat dublu	29
5.6.3	Marcajul de siguranță în modul privat	29
5.6.4	Programarea notificărilor periodice	30
5.6.5	Identificatorul de dispozitiv	30
5.7	Manual de implementare	31
5.7.1	Cerințe de sistem	31
5.7.2	Configurarea backend-ului	31
5.7.3	Schema bazei de date	32
5.7.4	Configurarea aplicației Android	33
5.8	Manual de utilizare	33
6	Testare și Evaluare	34
6.1	Metodologia de testare	34
6.2	Indicatori măsurați	34
6.3	Evoluția stării emoționale	35
6.4	Calitatea răspunsurilor și utilizabilitatea	36
6.5	Amenințări la validitate	36
7	Concluzii și Direcții Viitoare	38
	Bibliografie	41
A	Anexă - Utilizarea instrumentelor de inteligență artificială genera-	
	tivă	42
A.1	Detalii privind utilizarea DeepSeek	42
A.2	Detalii privind utilizarea Claude	43

Capitolul 1

Introducere

Sănătatea mintală reprezintă una dintre cele mai importante și, totodată, neglijate componente ale bunăstării umane. Într-o lume caracterizată printr-un ritm alert al vieții, presiune profesională crescută și incertitudine socială, un număr tot mai mare de persoane se confruntă cu anxietate, epuizare emoțională sau dificultăți în gestionarea propriilor trăiri. Conform Raportului Mondial privind Sănătatea Mintală publicat de Organizația Mondială a Sănătății în 2022, aproximativ un miliard de oameni trăiesc cu o tulburare mintală diagnosticabilă, iar accesul la servicii specializate rămâne limitat pentru majoritatea lor [1]. Barierele sunt multiple: costul ridicat al terapiei, reticența, rușinea socială asociată căutării ajutorului psihologic și lipsa disponibilității unui specialist [2].

Tehnologiile digitale au început să ocupe un loc din ce în ce mai important ca instrumente complementare de sprijin emoțional. Smartphone-urile, prezente astăzi la scară globală, oferă o platformă ideală pentru aplicații care pot fi accesate oricând, fără costurile sau logistica unei ședințe de terapie. Studii recente arată că aplicațiile conversaționale bazate pe inteligență artificială pot reduce simptomele de depresie și anxietate la utilizatorii care interacționează regulat cu ele [3]. Exemple precum Woebot sau Wysa au demonstrat că există atât cerere reală, cât și utilitate practică pentru astfel de soluții [4].

Aplicația dezvoltată în cadrul acestei lucrări, numită **MindBuddy**, propune o soluție adaptată contextului local: un companion AI empatic, care funcționează în limba română, capabil să întrețină conversații cu context persistent și să monitorizeze evoluția emoțională a utilizatorului în timp.

Contribuțiile principale

Principalele contribuții personale aduse în realizarea acestei lucrări sunt:

1. Proiectarea și implementarea integrală a arhitecturii client-server, incluzând aplicația Android, backend-ul Node.js și schema bazei de date PostgreSQL.
2. Implementarea sistemului de autentificare cu verificare prin email și resetare pa-

rolă, cu generare de token-uri criptografice și stocare securizată pe dispozitiv.

3. Integrarea API-ului DeepSeek pentru două sarcini distincte: generarea de răspunsuri empatică în limba română și clasificarea emoțiilor din mesajele utilizatorului.
4. Proiectarea și implementarea modului privat de conversație, în care conținutul mesajelor nu este salvat în baza de date, contextul fiind gestionat exclusiv în memoria dispozitivului, cu păstrarea exclusiv a marcajului de siguranță necesar modulului de detecție a crizei.
5. Implementarea modulului de vizualizare grafică a evoluției emoționale pe intervale de timp configurabile.
6. Proiectarea și implementarea unui mecanism de blocare locală a aplicației prin PIN de patru cifre cu hash criptografic (PBKDF2-SHA256 cu salt aleator) și deblocare biometrică opțională (amprentă sau recunoaștere facială), pentru protejarea conversațiilor față de accesul fizic al unor terți la dispozitiv.
7. Implementarea unui sistem de reminders locale prin WorkManager, cu patru frecvențe configurabile (o dată pe zi, de două ori, de patru ori sau din oră în oră) și mesaje variate de tip check-in care nu se repetă consecutiv, restricționate la intervalul 8:00–22:00.
8. Proiectarea unui modul de safety check care detectează automat mențiunile de gânduri suicidale prin LLM și prin pattern matching auxiliar, declanșează escaladare progresivă a sprijinului (reminders forțate la depășirea pragului de tristețe, blocaj total al conversațiilor la acumularea a patru mențiuni în șapte zile) și oferă două căi de deblocare: testul psihometric PHQ-2 și validarea de către un contact de încredere desemnat în prealabil.
9. Integrarea unui mecanism de persistență la nivel de dispozitiv pentru starea de criză, bazat pe `Settings.Secure.ANDROID_ID` și o tabelă dedicată în baza de date, care împiedică un utilizator să evite blocajul prin crearea unui cont nou pe același telefon.

Structura lucrării

Lucrarea este organizată în șase capitole. **Capitolul 2** descrie problema care a generat necesitatea aplicației și prezintă abordările existente în domeniu. **Capitolul 3** detaliază arhitectura sistemului. **Capitolul 4** prezintă funcționalitățile aplicației. **Capitolul 5** acoperă detaliile de implementare. Lucrarea se încheie cu **Capitolul 6**, care conține concluziile și direcțiile viitoare de dezvoltare.

Capitolul 2

Descrierea Problemei și Abordări Existente

2.1 Problema abordată

Accesul la sprijin pentru sănătatea mintală rămâne inegal distribuit la nivel global. Kessler și colaboratorii au arătat că aproape jumătate din populația adultă dezvoltă cel puțin o tulburare mintală diagnosticabilă de-a lungul vieții, însă perioada medie dintre apariția simptomelor și prima intervenție specializată este de 8 până la 10 ani [2]. În România, decalajul este și mai pronunțat: numărul psihologilor și psihiatrilor raportat la populație este semnificativ sub media europeană, iar costul ședințelor de psihoterapie îl face inaccesibil pentru o mare parte a populației.

Organizația Mondială a Sănătății definește sănătatea mintală ca o stare de bunăstare în care individul își realizează propriul potențial, poate face față stresurilor normale ale vieții și poate contribui productiv la comunitate [1]. Această definiție subliniază că sănătatea mintală nu înseamnă doar absența bolii, ci implică o funcționare activă și adaptativă, aspect pe care instrumentele digitale de suport îl pot sprijini prin monitorizare continuă și feedback imediat.

Problema concretă pe care o adresează aplicația MindBuddy este lipsa unui instrument digital de suport emoțional disponibil în limba română, cu funcționalitate completă de conversație cu context persistent și mecanisme de confidențialitate granulare. Utilizatorii care doresc să discute probleme emoționale se confruntă fie cu bariere de limbă (aplicațiile existente sunt predominant în engleză), fie cu lipsa controlului asupra datelor stocate.

La baza acestei lucrări stă și o experiență personală: în perioadele în care m-am confruntat cu anxietatea, am încercat să mă deschid față de cei apropiați, însă conversațiile rămâneau fără un răspuns clar, nu pentru că nu le păsa, ci pentru că nici eu nu știam cum să numesc ceea ce simțeam. Îmi lipsea un spațiu sigur în care să explorez aceste trăiri fără presiunea de a le explica perfect. De aici și împreună cu recomandările coordonatorului a pornit ideea MindBuddy: o aplicație care ascultă fără să judece, ajută utilizatorul să se cunoască mai bine și, tocmai prin această experiență de a fi înțeles, îl poate motiva să facă pasul către un specialist.

2.2 Procesarea limbajului natural și analiza sentimentelor

Procesarea limbajului natural (NLP) este ramura inteligenței artificiale care se ocupă cu înțelegerea și generarea textului uman de către sisteme computaționale. Una dintre aplicațiile sale cele mai relevante pentru contextul acestei lucrări este analiza sentimentelor, definită ca identificarea automată a opiniilor, emoțiilor sau atitudinilor exprimate într-un text [5].

Saltul calitativ major în domeniu a venit în 2017 odată cu publicarea arhitecturii Transformer de către Vaswani și colaboratorii în lucrarea *Attention Is All You Need* [6]. Mecanismul de atenție (*self-attention*) permite modelului să pondereze importanța fiecărui cuvânt din context față de celelalte, captând relații semantice pe distanțe lungi în text. Această arhitectură a stat la baza întregii familii moderne de modele mari de limbaj (LLM).

Modelele mari de limbaj nu necesită antrenare pe un set specific de date pentru fiecare sarcină, ci pot fi adaptate prin *prompting*: instrucțiuni în limbaj natural care ghidează comportamentul modelului. În cadrul acestei lucrări, API-ul DeepSeek [7] este utilizat în două roluri: generarea răspunsurilor conversaționale empaticе și clasificarea emoțiilor din mesajele utilizatorului.

2.3 Abordări existente și comparație

Piața aplicațiilor de suport emoțional bazate pe AI cunoaște câteva soluții notabile, fiecare cu o abordare distinctă. *Woebot* combină tehnici de terapie cognitiv-comportamentală (CBT) cu un chatbot conversațional și a demonstrat reduceri semnificative ale simptomelor de anxietate și depresie în studii clinice randomizate [3]. *Wysa* oferă un mix de CBT și mindfulness, cu accent pe gestionarea stresului zilnic. *Luxton* oferă o perspectivă cuprinzătoare asupra utilizării inteligenței artificiale în sănătatea comportamentală, subliniind atât potențialul, cât și riscurile acestor instrumente [4].

Torous și colaboratorii au analizat implicarea utilizatorilor cu aplicații de sănătate mintală pe smartphone, identificând că factori precum personalizarea, feedback-ul imediat și continuitatea conversațională sunt esențiali pentru retenția pe termen lung [8].

Tabelul 2.1: Comparație între aplicații existente de suport emoțional AI

Aplicație	Abordare	Limbă	Context	Mod privat	Safety
Woebot	CBT structurat	EN	Nu	Nu	Parțial
Wysa	CBT + mindfulness	EN	Parțial	Nu	Parțial
Replika	Companion AI	EN	Da	Nu	Nu
MindBuddy	LLM empatic	RO	Da	Da	Da

Legendă coloane: „Context” - păstrarea istoricului conversațiilor între sesiuni; „Mod privat” - modul de discuție în care conversațiile nu sunt stocate; „Safety” - sistem de detecție și escaladare în caz de gânduri suicidale.

Din analiza aplicațiilor existente reies trei lacune principale pe care MindBuddy le adresează: absența suportului nativ în limba română, lipsa unui mecanism de confidențialitate granular care să ofere utilizatorului control complet asupra datelor stocate și absența unui sistem proactiv de safety check care să escaladeze sprijinul când utilizatorul exprimă gânduri de auto-vătămare.

Capitolul 3

Arhitectura Aplicației

3.1 Arhitectura generală

Aplicația MindBuddy urmează o arhitectură client-server pe trei niveluri: nivelul de prezentare (aplicația Android), nivelul de logică aplicativă (backend Node.js) și nivelul de persistență a datelor (PostgreSQL). Figura 3.1 prezintă o vedere de ansamblu a sistemului.

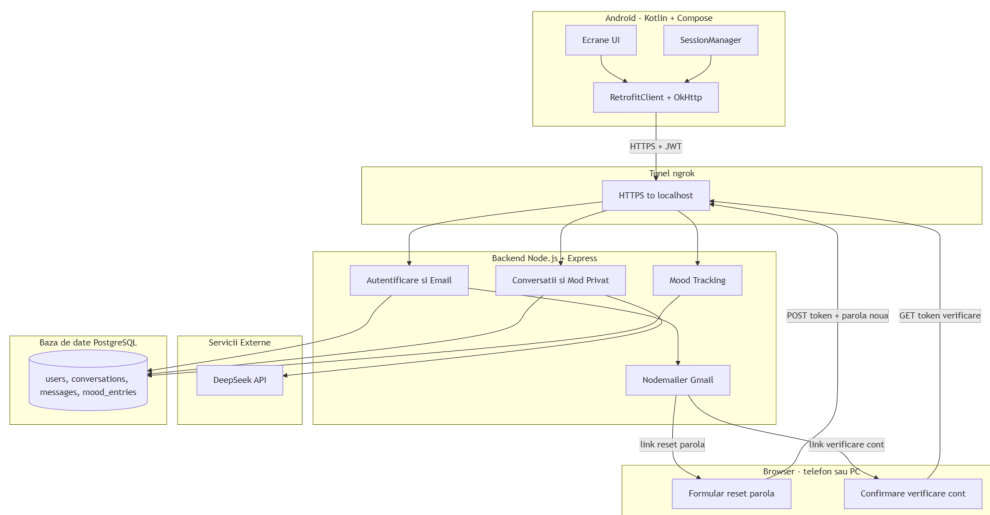


Figura 3.1: Arhitectura generală a sistemului MindBuddy

Comunicarea dintre client și server se realizează prin HTTP REST, securizat prin tunelul HTTPS oferit de ngrok [9] în mediul de dezvoltare. La fiecare cerere HTTP, un interceptor OkHttp atașează automat token-ul JWT din **SessionManager** și secretul aplicației în header-ele HTTP, înainte ca cererea să fie trimisă.

3.2 Componentele sistemului

3.2.1 Aplicația Android

Aplicația Android este construită cu Kotlin [10] și Jetpack Compose, framework-ul declarativ de UI recomandat de Google. Navigarea între ecrane este gestionată de `NavController` din Jetpack Navigation Compose, cu un graf de navigare definit în `MainActivity`. Comunicarea cu backend-ul este realizată prin `RetrofitClient`, un singleton care gestionează clientul HTTP și atașează automat credențialele la fiecare cerere.

Ecranele principale ale aplicației sunt:

- **LoginScreen** — autentificare, înregistrare, resetare parolă;
- **MainMenuScreen** — punct de intrare după autentificare;
- **ChatScreen** — interfața de conversație cu modul privat integrat;
- **ConversationHistoryScreen** — lista conversațiilor salvate;
- **EmotionChartScreen** — graficul evoluției emoționale;
- **PinScreen** / **PinSettingsScreen** / **PinResetScreen** — setarea, verificarea și resetarea PIN-ului de blocare;
- **RemindersSettingsScreen** — configurarea frecvenței notificărilor de check-in;
- **TrustedContactScreen** — desemnarea persoanei de încredere care poate debloca aplicația în caz de criză;
- **CrisisLockedScreen** — ecran de protecție afișat când sistemul detectează acumulare de gânduri suicidale, cu numere de urgență și opțiuni de deblocare.

3.2.2 Backend-ul Node.js

Backend-ul este implementat cu Node.js și Express.js [11], organizat în trei fișiere: `db.js` (configurarea pool-ului de conexiuni PostgreSQL), `auth.js` (middleware-ul de verificare JWT) și `index.js` (toate rutele și logica aplicației). Rutele sunt grupate pe domenii funcționale:

- `/auth/*` — înregistrare, autentificare, verificare email, resetare parolă;
- `/conversations/*` — creare, listare, mesaje, trimitere mesaj, ștergere;
- `/chat/private` — conversație privată fără stocare;
- `/mood/entries` — date pentru graficul de emoții;
- `/auth/pin-reset/*` — resetarea PIN-ului local prin cod trimis pe email;

- `/safety/*` — starea de safety a contului, deblocarea de criză prin PHQ-2 sau contact de încredere, verificarea blocajului la nivel de dispozitiv.

Un middleware global verifică prezența și valabilitatea token-ului JWT și a secretului aplicației pe toate rutele protejate, înainte ca cererea să ajungă la logica specifică rutei.

3.2.3 Baza de date PostgreSQL

Baza de date PostgreSQL [12] conține cinci tabele principale, cu relații de integritate referențială. Alegerea unei baze de date relaționale în locul uneia de tip document (precum MongoDB) a fost motivată de natura puternic structurată și interconectată a datelor: un utilizator are mai multe conversații, fiecare conversație are mai multe mesaje, iar fiecare mesaj normal generează o intrare emoțională. Aceste relații unu-la-mai-mulți se modelează natural prin chei străine, iar constrângerea `ON DELETE CASCADE` garantează integritatea referențială la ștergere - un mesaj nu poate exista fără conversația părinte. În plus, interogările pentru graficul emoțional (agregări și filtrări pe intervale de timp) sunt exact tipul de operații pentru care SQL este optimizat. O bază de date document ar fi adăugat o flexibilitate de care proiectul nu avea nevoie, în detrimentul garanțiilor de consistență.

Tabelul 3.1: Structura bazei de date

Tabelă	Coloane cheie	Scop
<code>users</code>	<code>id</code> , <code>username</code> , <code>email</code> , <code>password_hash</code> , <code>is_verified</code> , <code>verification_token</code> , <code>reset_token</code> , <code>pin_reset_code</code> , <code>trusted_contact_email</code> , <code>crisis_locked</code> , <code>suicidal_count</code>	Datele contului, token-urile de securitate și starea de safety
<code>conversations</code>	<code>id</code> , <code>user_id</code> , <code>title</code> , <code>created_at</code> , <code>updated_at</code>	Metadatele conversațiilor
<code>messages</code>	<code>id</code> , <code>conversation_id</code> , <code>role</code> , <code>content</code> , <code>created_at</code>	Conținutul efectiv al mesajelor
<code>mood_entries</code>	<code>id</code> , <code>user_id</code> , <code>happy</code> , <code>sad</code> , <code>anxious</code> , <code>angry</code> , <code>neutral</code> , <code>dominant_emotion</code> , <code>suicidal_mention</code> , <code>text_excerpt</code>	Scorurile emoționale per mesaj și marcarea mențiunilor suicidale
<code>device_crisis_locks</code>	<code>device_id</code> , <code>locked_at</code> , <code>unlocked_at</code>	Persistența blocajului de criză la nivel de dispozitiv, independent de cont

3.2.4 Servicii externe

DeepSeek API [7] este utilizat pentru două sarcini: generarea răspunsurilor conversaționale (modelul `deepseek-chat` cu temperatura 0.7) și clasificarea emoțiilor (același model cu temperatura 0, pentru determinism). Alegerea unui API extern în locul antrenării unui model propriu a fost motivată de calitatea superioară a unui model deja optimizat la scară largă și de resursele computaționale prohibitive necesare antrenării unui LLM propriu.

Nodemailer este folosit pentru trimiterea emailurilor de verificare a contului și de resetare a parolei, printr-un cont Gmail dedicat aplicației.

ngrok [9] creează un tunel HTTPS securizat de la un URL public la serverul local, permițând aplicației Android să comunice cu backend-ul în faza de dezvoltare.

3.3 Diagrama cazurilor de utilizare

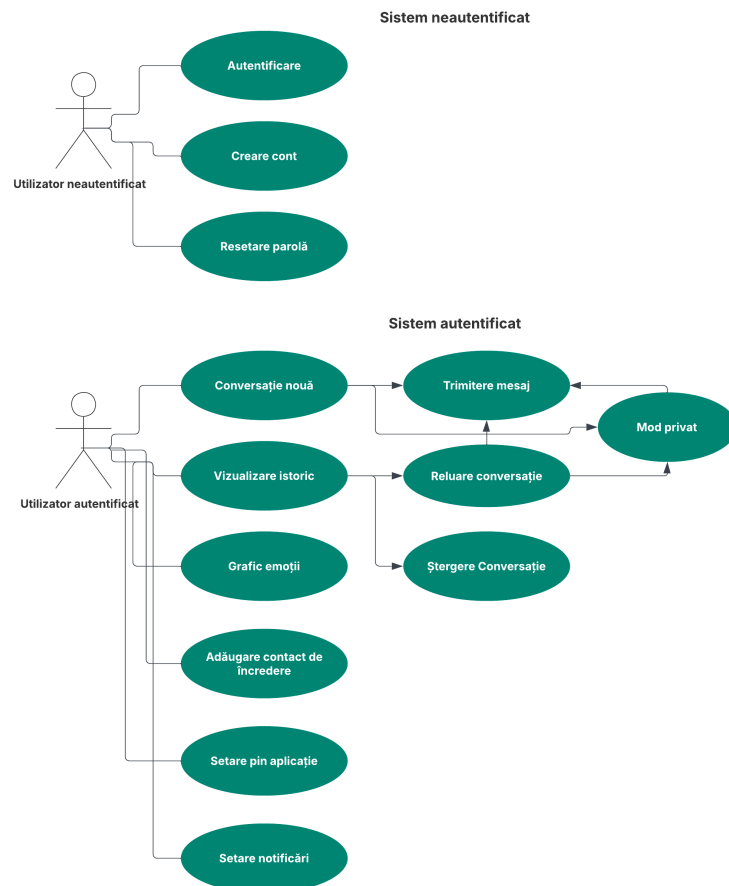


Figura 3.2: Diagrama cazurilor de utilizare MindBuddy

Sistemul are un singur tip de actor: utilizatorul. Înainte de autentificare, utilizatorul poate să se înregistreze, să se autentifice și să solicite resetarea parolei. După autentificare, are acces la toate funcționalitățile: inițierea și continuarea conversațiilor, activarea modului privat, ștergerea conversațiilor și vizualizarea graficului emoțional.

3.4 Diagrama de secvență — trimiterea unui mesaj

Trimiterea unui mesaj este fluxul principal al aplicației și implică cel mai mare număr de componente care comunică între ele. Diagrama din Figura 3.3 urmărește pas cu pas ce se întâmplă din momentul în care utilizatorul apasă pe butonul de trimitere până când primește răspunsul generat, trecând prin aplicația Android, backend-ul Node.js, baza de date PostgreSQL și API-ul DeepSeek.

Înainte de orice altceva, backend-ul verifică token-ul JWT și secretul aplicației, astfel încât doar cererile legitime să ajungă mai departe. Dacă mesajul este primul din conversație, conversația este creată direct din corpul cererii, fără un apel separat. Urmează încărcarea istoricului din baza de date, clasificarea emoției mesajului și salvarea acestuia împreună cu starea emoțională detectată. Abia după ce contextul complet este pregătit, backend-ul trimite cererea către API-ul DeepSeek, limitând istoricul pentru a nu depăși fereastra de context, iar răspunsul primit este salvat la rândul lui și returnat aplicației.

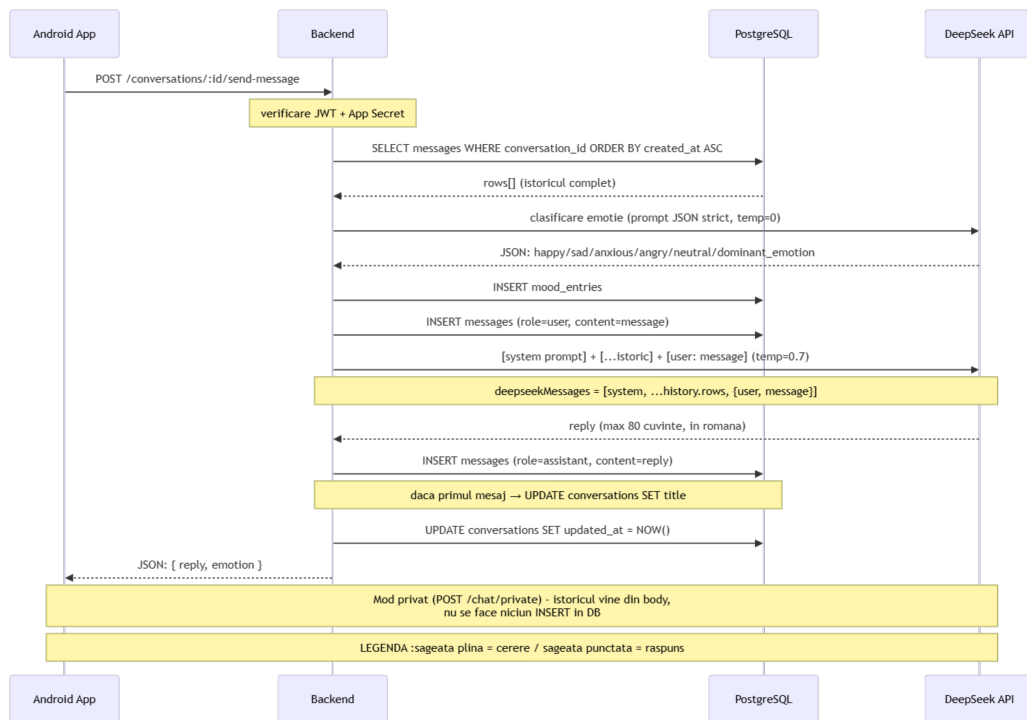


Figura 3.3: Diagrama de secvență pentru trimiterea unui mesaj

Capitolul 4

Funcționalitățile Aplicației

4.1 Autentificarea și gestionarea contului

4.1.1 Înregistrare și verificare prin email

Procesul de înregistrare implică introducerea unui nume de utilizator, a unei adrese de email și a unei parole. Sistemul validează datele, hashuiește (îi creează o amprentă criptografică) parola cu `bcrypt` [13] și generează un token de verificare de 64 de caractere hexazecimale folosind `crypto.randomBytes(32)`. Un email de activare este trimis automat, conținând un link care redirecționează utilizatorul într-un browser. Apăsarea linkului activează contul și permite autentificarea.

Această abordare în doi pași previne crearea de conturi cu adrese de email false și asigură că fiecare cont este asociat unui utilizator real.

4.1.2 Autentificarea și sesiunea JWT

La autentificare, utilizatorul poate folosi fie numele de utilizator, fie adresa de email. Sistemul caută înregistrarea în baza de date, compară parola introdusă cu hash-ul stocat și, dacă verificarea reușește, semnează și returnează un token JWT [14] cu durată de 7 zile. Token-ul este stocat pe dispozitiv în `EncryptedSharedPreferences` (AES-256-GCM), care criptează atât cheile cât și valorile la nivel de sistem de fișiere.

Valabilitatea token-ului este verificată local la pornirea aplicației, prin decodificarea payload-ului JWT din Base64 și compararea câmpului `exp` cu timpul curent, fără a face o cerere de rețea inutilă.

Spre deosebire de o sesiune clasică ținută pe server, token-ul JWT este autonom: poartă în el identitatea utilizatorului, semnată criptografic, astfel încât serverul validează cererile fără să mențină o stare de sesiune sau să interogheze baza de date la fiecare apel.

4.1.3 Resetarea parolei

Utilizatorul care și-a uitat parola introduce adresa de email în formularul dedicat. Dacă emailul există în baza de date, sistemul generează un token de resetare cu valabilitate de 15 minute și trimite un email cu un link. Link-ul deschide în browser o pagină HTML servită direct de backend, care conține un formular simplu pentru introducerea noii parole. Formularul trimite un POST la backend, care verifică tokenul, actualizează hash-ul parolei și invalidează tokenul.

4.2 Conversațiile cu context persistent

4.2.1 Crearea și gestionarea conversațiilor

Conversațiile sunt create lazy (creare amânată - până la nevoie/apăsare): la apăsarea butonului „Începe conversație”, aplicația navighează la ecranul de chat cu identificadorul special **"new"**. Nicio cerere de rețea nu este efectuată la acest pas. Prima intrare în baza de date este creată abia la trimiterea primului mesaj, evitând acumularea de conversații goale pentru utilizatorii care intră și ies fără să scrie nimic.

Titlul conversației este setat automat la primele 60 de caractere ale primului mesaj, urmat de „...” dacă mesajul este mai lung. Timestamp-ul `updated_at` este actualizat la fiecare mesaj nou, permițând ordonarea descrescătoare a istoricului după activitate recentă.

4.2.2 Contextul persistent: o strategie la nivel de aplicație

Modelele de limbaj accesate prin API sunt *fără stare*: fiecare cerere este independentă, iar modelul nu reține nimic între apeluri. Soluția nu este o memorie reală, ci o strategie a aplicației, care retrimite istoricul conversației la fiecare mesaj nou.

La primirea unui mesaj, backend-ul interoghează baza de date pentru toate mesajele anterioare ale conversației, ordonate cronologic. Acestea sunt mapate în formatul cerut de API, prefixate cu un mesaj de sistem care definește comportamentul asistentului în română și completate cu mesajul curent:

```
1 const deepseekMessages = [  
2   {  
3     role: "system",  
4     content: "Ești un companion empatic pentru sanatate mintala.  
5     Nu ești terapeut licentiat. Raspunde INTOTDEAUNA in limba  
6     romana. Oferă raspunsuri suportive (maxim 80 de cuvinte)."  
7   },  
8   ...history.rows.map(row => ({  
9     role: row.role,      // "user" sau "assistant"  
10    content: row.content  
11  })),  
12  { role: "user", content: message }  
13 ];
```

Listing 4.1: Construirea contextului complet pentru DeepSeek

Astfel, în cadrul unei conversații, asistentul pare să „își amintească” discuția, deși primește de fiecare dată istoricul reasamblat, plafonat de fereastra de context.

4.2.3 Istoricul conversațiilor

Ecranul de istoric afișează toate conversațiile utilizatorului, sortate descrescător după ultima actualizare. Fiecare conversație este afișată cu titlul și data. La selectarea unei conversații, aplicația încarcă toate mesajele din baza de date și le afișează în ordine cronologică, reconstituind dialogul complet. Ștergerea unei conversații necesită confirmare explicită și elimină automat toate mesajele.

4.3 Modul privat

Modul privat adresează nevoia utilizatorilor de a discuta subiecte sensibile fără ca acestea să fie înregistrate. Funcționalitatea este accesibilă printr-o iconiță de lacăt din header-ul ecranului de chat. La activare, header-ul devine roșiatic și apare un mesaj de confirmare centrat, vizibil pentru utilizator.

În modul privat, la fiecare mesaj trimis, aplicația construiește lista completă a mesajelor din sesiunea curentă și o include în cererea `POST /chat/private`. Backend-ul nu salvează conversația: nu creează conversații, nu reține mesajele și nu înregistrează profilul emoțional. Singura excepție este stratul de siguranță, care păstrează un mar-caj minimal atunci când detectează o mențiune suicidală, descris în Secțiunea 4.7.6.

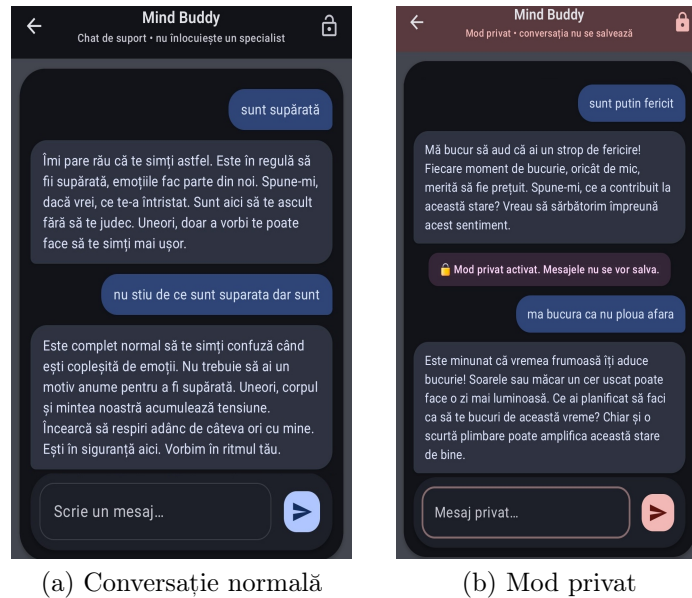


Figura 4.1: Interfața de chat în mod normal (a) și în mod privat (b)

4.4 Modul de analiză a emoțiilor

La fiecare mesaj trimis în mod normal (nu privat), sistemul clasifică și stochează automat emoțiile conținute în text. Clasificarea este realizată de un apel separat la API-ul DeepSeek, cu un prompt strict care solicită returnarea unui obiect JSON cu distribuția procentuală a cinci emoții: fericire, tristețe, anxietate, furie și neutralitate, cu suma exactă de 100. Răspunsul este validat înainte de a fi acceptat.

Rezultatele sunt stocate în tabela `mood_entries` și pot fi vizualizate în ecranul „Grafic Emoții”, care afișează evoluția emoțională pe trei intervale de timp: ziua curentă, ultimele 7 zile și ultimele 30 de zile. Utilizatorul poate filtra graficul pentru a afișa o emoție anume sau toate emoțiile simultan.

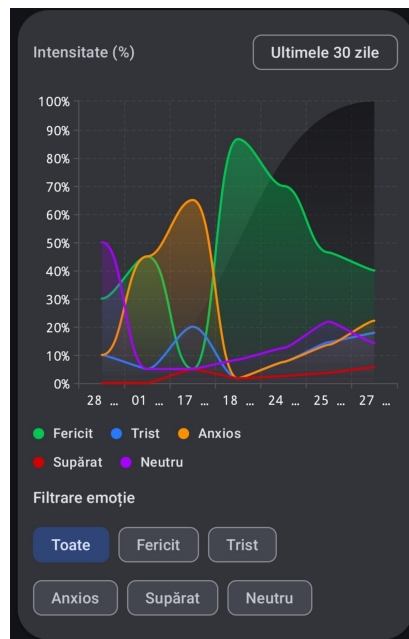


Figura 4.2: Graficul de Emoții al utilizatorului în ultima lună

4.5 Blocarea aplicației cu PIN și amprentă

Conversațiile despre stres, anxietate sau gânduri intime pot conține informații pe care utilizatorul nu dorește să le expună unor terți care i-ar putea accesa telefonul. Pentru a reduce această vulnerabilitate, aplicația implementează un mecanism propriu de blocare prin PIN de patru cifre, independent de blocarea sistemului de operare.

PIN-ul nu este niciodată stocat în clar și nu părăsește dispozitivul. La setare, se generează un `salt` aleator de 16 octeți (o valoare aleatoare adăugată înainte de hashing), iar PIN-ul este derivat printr-o funcție PBKDF2-SHA256 cu 120.000 de

iterații, rezultând un hash de 256 de biți care este apoi stocat în *EncryptedSharedPreferences*. La verificare, PIN-ul introdus trece prin aceeași derivare, iar comparația cu hash-ul salvat se face în timp constant pentru a împiedica atacuri de tip timing.

Aplicația cere reintroducerea PIN-ului de fiecare dată când iese și revine în prim-plan, prin observarea `ProcessLifecycleOwner`-ului întregii aplicații. Deblocarea biometrică (amprentă sau recunoaștere facială) este oferită opțional ca alternativă rapidă la PIN, prin `BiometricPrompt` cu autentificatori clasificați `BIOMETRIC_STRONG`.

În cazul în care utilizatorul uită PIN-ul, ștergerea contului nu este o opțiune (ar duce la pierderea istoricului). În schimb, aplicația permite o resetare în doi pași: solicitarea unui cod numeric de șase cifre trimis pe email-ul contului, urmată de validarea codului împreună cu parola contului. Doar după această dublă confirmare hash-ul PIN-ului local este șters, iar utilizatorul poate seta unul nou. PIN-ul în sine nu este niciodată cunoscut serverului.

4.6 Reminders și notificări locale

Studiile asupra implicării cu aplicații de sănătate mintală arată că un factor major în retenție este existența unor solicitări regulate de check-in [8]. MindBuddy implementează un sistem de reminders prin notificări locale, independent de server.

Utilizatorul alege una dintre cinci frecvențe, de la dezactivat până la din oră în oră, fiecare cu un set fix de ore (de exemplu, de două ori pe zi declanșează notificări la 10 și 18). Toate sunt restricționate la intervalul 8:00–22:00, pentru a nu perturba odihna.

Implementarea folosește `WorkManager` [15] cu un `PeriodicWorkRequest` care se trezește la fiecare oră și verifică dacă ora curentă se află în setul programat. Față de un `AlarmManager` clasic, oferă reziliență la restart-ul dispozitivului și nu necesită permisiuni speciale pe Android 12+.

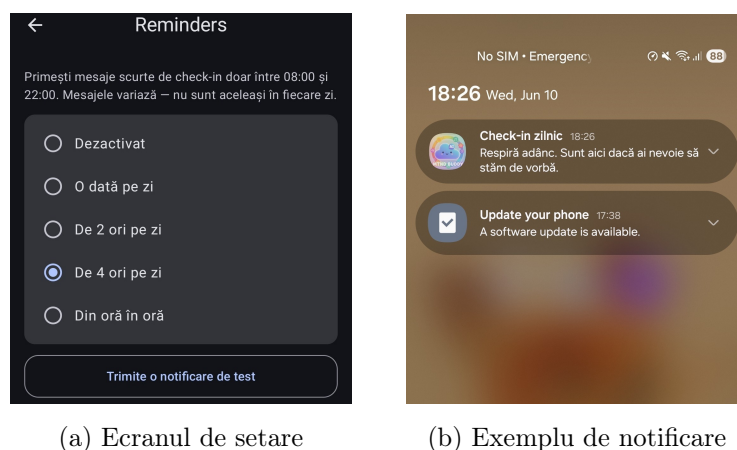


Figura 4.3: Notificări (Reminders) și setarea acestora

Pentru a evita monotonia, sistemul nu trimite același mesaj de două ori la rând: există un set de paisprezece variante de check-in (de exemplu, „Hey, cum te simți astăzi?”, „Pauză de gânduri? Hai să le punem în ordine împreună.”) și un identificator persistent al ultimului mesaj trimis, folosit pentru a alege o variantă diferită la următoarea notificare.

4.7 Modulul de safety check (verificare de siguranță)

Cea mai delicată parte a aplicației este modulul care detectează automat semnele de criză și escaladează sprijinul oferit. Sistemul funcționează pe două niveluri de severitate, fiecare cu un răspuns distinct.

4.7.1 Pragul „îngrijorător”: reminders forțate

La fiecare mesaj trimis în mod normal, backend-ul calculează media procentului *sad* pe ultimele cinci mesaje stocate. Dacă media depășește 50%, sistemul intră în starea *concerning* și forțează frecvența de notificări la „din oră în oră”, indiferent de preferința setată anterior de utilizator. Pragul de 50% este o valoare aleasă empiric, un compromis între a reacționa prea devreme și a rata o perioadă grea, nu un prag derivat din date clinice. Această escaladare proactivă oferă utilizatorului ocazii mai dese de a discuta despre ce simte, fără a-l obliga să acționeze. Când media revine sub prag, aplicația restaurează frecvența originală.

4.7.2 Pragul „criză”: blocajul conversațiilor

Pentru detecția gândurilor suicidale am extins promptul către DeepSeek cu un câmp suplimentar *suicidal_mention* de tip boolean, care marchează mesajele care exprimă dorința de auto-vătămare, de a muri sau de a-și încheia viața. Pentru robustețe, există și un strat de pattern matching (potrivire de tipare) local cu expresii regulate adaptate limbii române (de exemplu, „vreau să mor”, „să mă arunc”, „nu mai vreau să trăiesc”), care prinde mesajele evidente chiar dacă apelul către LLM eșuează. Cele două detecții sunt combinate prin SAU logic.

La acumularea a patru mențiuni suicidale în ultimele șapte zile, sistemul activează modul de criză: utilizatorul este redirectionat către un ecran dedicat care afișează numere de urgență (TelVerde Antisucid 0800 801 200, Telefonul Copilului 116 111, 112), iar toate cererile de chat ulterioare primesc HTTP 423 (*Locked*) și nu mai sunt procesate. În același moment, un email automat este trimis către contactul de încredere al utilizatorului, semnalând că persoana are nevoie de sprijin. Și aici, valorile de patru mențiuni și șapte zile sunt alese empiric: destul cât un episod izolat să nu blocheze accidental aplicația, dar o acumulare reală să fie sesizată.

4.7.3 Deblocarea în două căi

După un *cooldown* obligatoriu de 24 de ore (perioadă în care utilizatorul nu poate face nimic în aplicație în afara apelării numerelor de urgență) sunt deschise două căi de deblocare. Prima este completarea testului psihometric PHQ-2 [16], un instrument validat clinic cu doar două întrebări scorate de la 0 la 3 fiecare. Un scor total sub 3 deblochează automat aplicația; un scor mai mare redirecționează utilizatorul spre a doua cale.

A doua cale implică desemnarea unui contact de încredere (părinte, prieten apropiat, terapeut) pe care utilizatorul l-a configurat în prealabil. La cerere, aplicația trimite un email cu un link unic și cu valabilitate de 24 de ore. Persoana de încredere, prezentă fizic lângă utilizator, accesează link-ul și confirmă că utilizatorul este în siguranță, deblocând aplicația. Acest mecanism transferă responsabilitatea finală a deciziei către un alt om, evitând atât blocarea permanentă, cât și autovalidarea care ar anula scopul protecției.

4.7.4 Contactul de încredere obligatoriu

Pentru ca mecanismul de criză să fie funcțional, fiecare cont trebuie să aibă un contact de încredere setat. Backend-ul respinge orice cerere de chat cu HTTP 428 (*Pre-condition Required*) dacă utilizatorul nu a desemnat încă o persoană, iar aplicația afișează un banner permanent pe ecranul principal care îl invită să facă acest pas. Astfel, niciun cont nu poate ajunge într-o situație de criză fără o cale de revenire pregătită din timp.

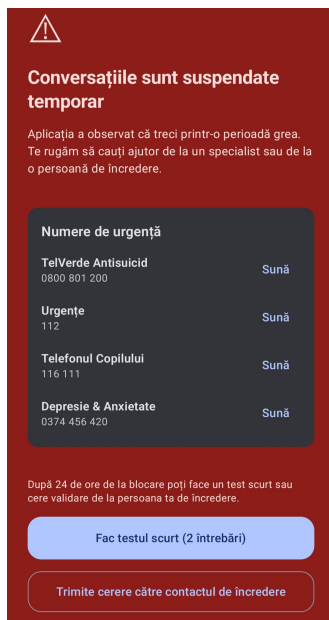


Figura 4.4: Ecranul de criză: numere de urgență și cele două căi de deblocare.

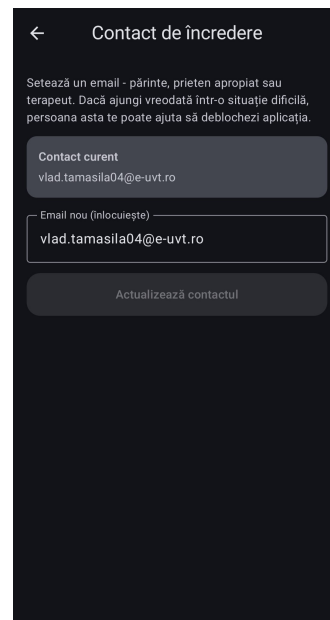


Figura 4.5: Ecranul de adăugare a contactului de încredere.

4.7.5 Persistența la nivel de dispozitiv

Un utilizator aflat în criză ar putea încerca să ocolească blocajul ștergând conținutul și creând altul pe același telefon. Pentru a împiedica acest lucru, fiecare instalare generează un identificator stabil derivat din *Settings.Secure.ANDROID_ID* (cu fallback pe un UUID salvat în *EncryptedSharedPreferences*), trimis în antetul X-Device-Id la fiecare cerere. Când un cont intră în criză, serverul îl reține în tabela *device_crisis_locks*, iar la pornire orice cont, vechi sau nou, este verificat împotriva ei; un dispozitiv blocat menține starea de criză indiferent de cont.

4.7.6 Detecția crizei în modul privat

Modul privat ridică o întrebare de siguranță: dacă nimic nu se salvează, un utilizator în criză ar putea ocoli protecția activând lacătul. De aceea, modul privat păstrează stratul de detecție într-o formă minimală: textul este clasificat ca în modul normal, dar rezultatul se stochează doar dacă apare o mențiune suicidală. În acel caz, în *mood_entries* se inserează un rând cu marcajul de siguranță (*suicidal_mention* = TRUE), fără text și fără profil emoțional. Astfel conținutul rămâne neînregistrat, însă pragul de patru mențiuni în șapte zile funcționează la fel, iar blocajul de criză și alerta către contactul de încredere se declanșează identic cu modul normal.

4.8 Interfața cu utilizatorul

Interfața respectă ghidurile Material Design 3 și se adaptează automat la tema light/dark a sistemului. Toate ecranele au indicator de încărcare la operațiile asincrone, mesaje clare de eroare și confirmare pentru acțiunile distructive (ștergerea conversațiilor). În plus, header-ul chatului afișează permanent avertismentul „*Chat de suport • nu înlocuiește un specialist*”, subliniind natura complementară a instrumentului.



Figura 4.6: Ecranul Principal

Capitolul 5

Detalii de Implementare

5.1 Tehnologii și instrumente utilizate

- **Kotlin + Jetpack Compose** [10] — limbajul și framework-ul oficial Google pentru Android, cu suport nativ pentru corutine și UI declarativ.
- **Retrofit 2 + OkHttp** [17] — client HTTP type-safe cu interceptori pentru autentificare automată.
- **Node.js + Express.js** [11] — backend asincron cu model event-loop, potrivit pentru operații I/O frecvente.
- **PostgreSQL** [12] — bază de date relațională cu suport pentru UUID, constrângeri de integritate și CASCADE.
- **DeepSeek API** [7] — model mare de limbaj pentru generarea răspunsurilor și clasificarea emoțiilor.
- **JWT** [14] — autentificare stateless (fără stare) prin token semnat criptografic.
- **bcrypt** [13] — hashing adaptiv pentru parole, rezistent la atacuri prin forță brută.
- **Nodemailer** — trimitere emailuri prin Gmail SMTP.
- **ngrok** [9] — tunel HTTPS pentru expunerea serverului local în faza de dezvoltare.

5.2 Implementarea autentificării

5.2.1 Hashing-ul parolelor

Parolele nu sunt niciodată stocate în clar. Algoritmul bcrypt cu 10 runde de salt generează un hash unic pentru fiecare parolă, chiar dacă două parole sunt identice. Factorul de cost adaptiv al bcrypt permite creșterea numărului de runde pe măsură ce hardware-ul devine mai performant, menținând rezistența la atacuri:

```
1 const hash = await bcrypt.hash(password, 10);
```

Listing 5.1: Hashing-ul parolei la înregistrare

5.2.2 Generarea și validarea JWT

La autentificare, serverul generează un token JWT, un șir de caractere semnat criptografic care conține informații despre utilizator: username, email și un identificator unic. Acest token este trimis aplicației Android și salvat criptat pe dispozitiv. La fiecare pornire a aplicației, tokenul este verificat local. Aplicația îl decodifică și verifică data de expirare fără a contacta serverul, economisind o cerere de rețea inutilă:

```
1 val payloadJson = String(
2     Base64.decode(parts[1], Base64.URL_SAFE or Base64.NO_WRAP)
3 )
4 val payload = JSONObject(payloadJson)
5 val expSec = payload.optLong("exp", 0L)
6 val nowSec = System.currentTimeMillis() / 1000
7 return expSec > nowSec + 5 // buffer de 5 secunde
```

Listing 5.2: Verificarea locală a token-ului JWT

5.3 Interceptorul de autentificare OkHttp

Toate cererile HTTP sunt procesate de un interceptor OkHttp care atașează automat token-ul JWT și secretul aplicației. Această abordare centralizează logica de autentificare, evitând duplicarea codului în fiecare apel API:

```
1 val client = OkHttpClient.Builder()
2     .addInterceptor { chain ->
3         val builder = chain.request().newBuilder()
4             .addHeader("X-App-Secret", APP_SECRET)
5         val token = session.getToken()
6         if (!token.isNullOrEmpty()) {
7             builder.addHeader("Authorization", "Bearer_$token")
8         }
9         chain.proceed(builder.build())
10     }
11     .build()
```

Listing 5.3: Interceptorul OkHttp pentru autentificare

5.4 Clasificarea emoțiilor

Clasificarea emoțiilor se face printr-o instrucțiune trimisă modelului, care îl obligă să răspundă exclusiv într-un format structurat și verificabil (JSON). Răspunsul primit este analizat și verificat înainte de a fi acceptat:

```

1  const keys = ["happy", "sad", "anxious", "angry", "neutral"];
2  for (const k of keys) {
3      const v = parsed[k];
4      if (!Number.isInteger(v) || v < 0 || v > 100)
5          throw new Error("Invalid_emotion_scores");
6  }
7  const sum = keys.reduce((acc, k) => acc + parsed[k], 0);
8  if (sum !== 100) throw new Error("Emotion_sum_must_be_100");
9  if (!keys.includes(parsed.dominant_emotion))
10     throw new Error("Invalid_dominant_emotion");

```

Listing 5.4: Validarea răspunsului de clasificare a emoțiilor

Dacă oricare dintre validări eșuează, cererea returnează eroare și datele emoționale nu sunt stocate, prevenind înregistrarea unor valori incorecte în baza de date.

5.5 Securitatea datelor

Securitatea a fost tratată ca o cerință transversală. OWASP Mobile Security [18] recomandă stocarea datelor sensibile pe dispozitiv exclusiv în zone criptate, iar MindBuddy respectă această cerință prin `EncryptedSharedPreferences`.

Toate rutele protejate verifică simultan token-ul JWT (identitatea utilizatorului) și header-ul `X-App-Secret` (identitatea aplicației client). Token-urile de verificare email și resetare parolă sunt generate cu `crypto.randomBytes(32)`, producând șiruri de 64 de caractere hexazecimale criptografic sigure. Token-ul de resetare are o fereastră de valabilitate de 15 minute și este invalidat imediat după utilizare.

5.6 Implementarea modulelor de protecție și siguranță

5.6.1 PIN-ul local cu derivare PBKDF2

PIN-ul utilizatorului este protejat prin derivare de cheie, nu prin stocarea directă a unui hash. La setare, se generează un salt aleator și se aplică PBKDF2 cu funcția HMAC-SHA256:

```

1  private fun pbkdf2(pin: String, salt: ByteArray): ByteArray {
2      val spec = PBEKeySpec(pin.toCharArray(), salt, ITERATIONS,
3                              KEY_BITS)
4      val factory = SecretKeyFactory.getInstance("
5          PBKDF2WithHmacSHA256")
6      return factory.generateSecret(spec).encoded
7  }
8  // ITERATIONS = 120_000, KEY_BITS = 256, SALT_BYTES = 16

```

Listing 5.5: Derivarea hash-ului PIN cu PBKDF2

Compararea la verificare se face în timp constant pentru a împiedica un atacator să deducă PIN-ul prin măsurarea timpului de execuție al funcției de comparație.

5.6.2 Detecția mențiunilor suicidale cu strat dublu

Pentru a evita situațiile în care un mesaj critic ar putea fi ignorat din cauza unui răspuns invalid de la LLM, clasificatorul este completat cu un strat local bazat pe pattern matching:

```
1 function looksSuicidal(text) {  
2   const t = String(text).toLowerCase();  
3   const patterns = [  
4     /vreau sa mor\b/, /vreau sa ma sinucid/,  
5     /sa ma omor/, /sa\-*mi iau viata/,  
6     /nu mai vreau sa traiesc/, /sa ma arunc(?: de| pe| in| din)/,  
7     /\bsuicid\b/, /kill myself/, /want to die/, /end my life/  
8   ];  
9   return patterns.some(p => p.test(t));  
10 }
```

Listing 5.6: Detecția keyword-based ca safety net

Rezultatul final este obținut prin combinarea logică *OR* a flag-ului întors de DeepSeek cu rezultatul funcției `looksSuicidal`, garantând că o frază evidentă nu poate trece neobservată chiar dacă API-ul extern eșuează sau dă un răspuns ambiguu. Întreaga funcție de clasificare este protejată cu `try/catch` și returnează un obiect implicit cu emoții neutre și `suicidal_mention` calculat doar din keywords, asigurând că procesarea mesajului nu se întrerupe niciodată.

Fiind bazat pe cuvinte-cheie, stratul de pattern matching poate scăpa formulări indirecte (fals-negative) sau se poate declanșa pe expresii figurate ori ironice, precum „mor de răs” sau „mă omoară examenul” (fals-pozitive). Primele sunt atenuate de combinarea cu LLM-ul, care prinde reformulările ratate de expresiile regulate, iar pentru cele din urmă blocajul se declanșează abia la patru mențiuni în șapte zile, astfel încât o potrivire izolată să nu aibă consecințe reale.

5.6.3 Marcajul de siguranță în modul privat

Ruta `/chat/private` nu scrie nimic în baza de date în fluxul normal. Singura excepție apare atunci când clasificarea întoarce o mențiune suicidală: în acel moment se inserează un rând minimal în `mood_entries`, fără excerpt și cu emoțiile lăsate neutre, suficient cât contorul de criză să rămână funcțional fără a stoca conținutul conversației:

```
1 const emo = await classifyEmotion(message);  
2 if (emo.suicidal_mention === true) {  
3   // rand minimal: doar flag-ul de siguranta, fara text  
4   await pool.query(  
5     `INSERT INTO mood_entries (message, mood, suicidal_mention) VALUES`  
6     + `('${message}', 'neutru', true);`  
7   );  
8 }
```

```

5      'INSERT INTO mood_entries
6      (user_id, happy, sad, anxious, angry, neutral,
7      dominant_emotion, text_excerpt, suicidal_mention)
8      VALUES ($1, 0, 0, 0, 0, 100, 'neutral', NULL, TRUE)',
9      [req.user.id]
10     );
11     const count = await countRecentSuicidalMentions(req.user.id);
12     if (count >= 4) {
13         await lockUserCrisis(req.user.id, req.headers["x-device-id"]);
14         // raspuns 423, la fel ca in modul normal
15     }
16 }

```

Listing 5.7: Marcajul minimal de siguranță în modul privat

Inserarea unui rând cu emoții neutre are un efect secundar minor: el intră și în media **sad** folosită de pragul „îngrijorător”, diluând-o ușor. Cum acest lucru se întâmplă doar la mesaje care conțin deja o mențiune suicidală - caz în care pragul de criză primează oricum - compromisul a fost considerat acceptabil.

5.6.4 Programarea notificărilor periodice

Reminders-ele folosesc **WorkManager**-ul Android, care oferă reziliență la restart-ul dispozitivului și nu necesită permisiunea **SCHEDULE_EXACT_ALARM** introdusă în Android 12:

```

1  val req = PeriodicWorkRequestBuilder<ReminderWorker>(1, TimeUnit.
    HOURS)
2      .setInitialDelay(1, TimeUnit.MINUTES)
3      .build()
4  WorkManager.getInstance(context.applicationContext)
5      .enqueueUniquePeriodicWork(
6      WORK_NAME,
7      ExistingPeriodicWorkPolicy.UPDATE,
8      req
9  )

```

Listing 5.8: Programarea worker-ului periodic

Worker-ul se trezește o dată pe oră și verifică trei condiții înainte de a trimite notificarea: frecvența nu este dezactivată, ora curentă se află în intervalul programat și nu s-a trimis deja o notificare în ultimele 50 de minute (anti-duplicat).

5.6.5 Identificatorul de dispozitiv

Pentru ca starea de criză să persiste între conturile create pe același telefon, aplicația generează la prima rulare un identificator stabil pe care îl atașează ca antet HTTP **X-Device-Id** la fiecare cerere:

```

1 val androidId = Settings.Secure.getString(
2     ctx.contentResolver, Settings.Secure.ANDROID_ID
3 )
4 val id = if (!androidId.isNullOrBlank() &&
5     androidId != "9774d56d682e549c") {
6     "aid_$androidId"
7 } else {
8     "uuid_" + UUID.randomUUID().toString()
9 }

```

Listing 5.9: Generarea identicatorului de dispozitiv

Valoarea 9774d56d682e549c este filtrată pentru că reprezintă un bug istoric al unor versiuni Android care întorceau același ID pe mai multe dispozitive. În aceste cazuri, fallback-ul pe UUID local oferă tot un identicator unic, salvat în *EncryptedSharedPreferences* pentru a supraviețui restart-urilor aplicației.

5.7 Manual de implementare

5.7.1 Cerințe de sistem

- **Backend:** Node.js 18+, PostgreSQL 14+, acces la internet pentru ngrok și DeepSeek API.
- **Android:** Android Studio Hedgehog sau mai nou, SDK Android 26+, dispozitiv sau emulator Android 8.0+.

5.7.2 Configurarea backend-ului

Backend-ul necesită un fișier `.env` cu variabilele de mediu:

```

1 PGHOST=localhost
2 PGPORT=5432
3 PGDATABASE=mental_health_db
4 PGUSER=your_user
5 PGPASSWORD=your_password
6 JWT_SECRET=your_jwt_secret
7 JWT_EXPIRES=7d
8 APP_SECRET=MentalHealthBot
9 DEEPSEEK_API_KEY=your_deepseek_key
10 EMAIL_USER=your_gmail@gmail.com
11 EMAIL_PASS=your_app_password
12 APP_BASE_URL=https://your-ngrok-url.ngrok-free.app
13 PORT=80

```

Listing 5.10: Variabile de mediu necesare (.env)

Pornirea backend-ului: `node index.js`. Pornirea tunelului: `ngrok http 80`.

5.7.3 Schema bazei de date

```
1 CREATE TABLE users (  
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3     username TEXT UNIQUE NOT NULL,  
4     email TEXT UNIQUE NOT NULL,  
5     password_hash TEXT NOT NULL,  
6     is_verified BOOLEAN DEFAULT FALSE,  
7     verification_token TEXT,  
8     reset_token TEXT,  
9     reset_token_expires TIMESTAMP,  
10    created_at TIMESTAMPTZ DEFAULT NOW(),  
11    pin_reset_code TEXT,  
12    pin_reset_expires TIMESTAMP,  
13    suicidal_count INT DEFAULT 0,  
14    last_suicidal_at TIMESTAMP,  
15    crisis_locked BOOLEAN DEFAULT FALSE,  
16    crisis_locked_at TIMESTAMP,  
17    crisis_unlocked_at TIMESTAMP,  
18    trusted_contact_email TEXT,  
19    crisis_unlock_token TEXT,  
20    crisis_unlock_token_expires TIMESTAMP  
21 );  
22  
23 CREATE TABLE conversations (  
24     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
25     user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
26     title VARCHAR(120) DEFAULT 'Conversatie noua',  
27     created_at TIMESTAMP DEFAULT NOW(),  
28     updated_at TIMESTAMP DEFAULT NOW()  
29 );  
30  
31 CREATE TABLE messages (  
32     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
33     conversation_id UUID NOT NULL  
34     REFERENCES conversations(id) ON DELETE CASCADE,  
35     role VARCHAR(10) NOT NULL CHECK (role IN ('user', 'assistant')),  
36     content TEXT NOT NULL,  
37     created_at TIMESTAMP DEFAULT NOW()  
38 );  
39  
40 CREATE TABLE mood_entries (  
41     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
42     user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
43     happy INTEGER NOT NULL,  
44     sad INTEGER NOT NULL,  
45     anxious INTEGER NOT NULL,  
46     angry INTEGER NOT NULL,  
47     neutral INTEGER NOT NULL,  
48     dominant_emotion TEXT NOT NULL,  
49     text_excerpt TEXT,  
50     created_at TIMESTAMPTZ DEFAULT NOW(),
```

```

51     suicidal_mention BOOLEAN DEFAULT FALSE
52 );
53
54 CREATE TABLE device_crisis_locks (
55     device_id TEXT PRIMARY KEY,
56     locked_at  TIMESTAMP NOT NULL,
57     unlocked_at  TIMESTAMP
58 );

```

Listing 5.11: Script de creare a tabelor principale

5.7.4 Configurarea aplicației Android

URL-ul ngrok și secretul aplicației se configurează în `RetrofitClient.kt`:

```

1 private const val BASE_URL = "https://your-ngrok-url.ngrok-free.app/"
2 private const val APP_SECRET = "MentalHealthBot"

```

Listing 5.12: Configurarea URL-ului de bază în `RetrofitClient`

5.8 Manual de utilizare

La prima deschidere, aplicația parcurge o configurare făcută o singură dată: creare cont cu verificare prin email, setarea unui PIN (cu deblocare biometrică opțională) și desemnarea unui contact de încredere. Apoi, meniul principal devine punctul din care utilizatorul ajunge la toate funcționalitățile, așa cum arată Figura 5.1.

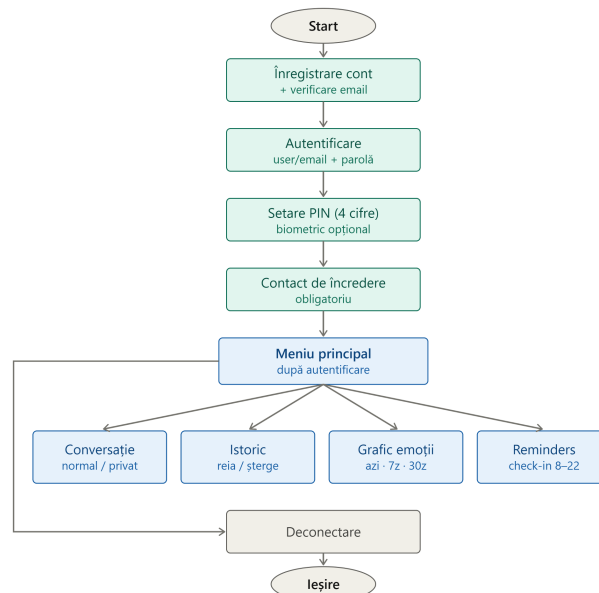


Figura 5.1: Fluxul de utilizare a aplicației MindBuddy

Capitolul 6

Testare și Evaluare

Pentru a verifica dacă aplicația funcționează nu doar tehnic, ci și din perspectiva celui care o folosește, am realizat o evaluare cu utilizatori reali, pe parcursul mai multor luni. Nu este un studiu clinic și nici nu își propune să fie, însă, spre deosebire de o simplă probă, am urmărit indicatori măsurabili din două categorii: indicatori **tehnici**, obținuți direct din sistem (acuratețe, recall, latență), și indicatori legați de **starea emoțională** a utilizatorului, extrași din datele pe care aplicația le înregistrează oricum în tabela `mood_entries`.

6.1 Metodologia de testare

La evaluare au participat cinci utilizatori, prieteni și colegi cu vârste apropiate de a mea, fiecare folosind propriul cont, în mod real, pe parcursul a aproximativ trei luni. Activitatea a variat mult de la o zi la alta, însă, în medie, fiecare a schimbat în jur de cinci mesaje pe zi, acumulând între 320 și 470 de mesaje, aproximativ 1.900 în total. Separat, pentru scenariile sensibile (modul privat și pragul de criză), am folosit eu trei conturi de test dedicate, ca să le pot verifica fără a expune date personale reale.

Pentru ca rezultatele să fie verificabile, fiecare indicator a fost măsurat printr-o procedură clară. Acuratețea clasificării emoțiilor am evaluat-o pe un eșantion de 100 de mesaje etichetate manual de trei adnotatori independenți. Detecția mențiunilor suicidale am testat-o separat, pe un set construit de 100 de mesaje: 40 cu formulări explicite de auto-vătămare și 60 neutre, rulând ambele straturi (LLM și pattern matching) și numărând detecțiile corecte și false. Evoluția stării emoționale am obținut-o agregând scorurile din `mood_entries`: pentru fiecare utilizator am calculat media procentului de emoții negative (tristețe plus anxietate) în primele două săptămâni și în ultimele două.

6.2 Indicatori măsurați

Tabelul 6.1 sintetizează rezultatele. Latența a fost dominată de apelul către DeepSeek; partea de procesare locală (interogare bază de date, validare) a rămas constant

sub 100 ms.

Tabelul 6.1: Indicatori măsurați în evaluare (5 utilizatori, ≈ 3 luni, ≈ 1.900 de mesaje)

Indicator	Mod de măsurare	Rezultat
Acuratețea clasificării emoțiilor	model vs. 100 mesaje etichetate	$\approx 63\%$
Acord între cei trei adnotatori	100 de mesaje	$\approx 79\%$
Recall la detecția mențiunilor suicidale	40 de formulări critice	$\approx 92\%$ (37/40)
Fals-pozitive la detecția de criză	60 de mesaje neutre	4/60
Latența medie a unui răspuns	incl. apelul DeepSeek	$\approx 3,4$ s
Rata erorilor de cerere	toate cererile din testare	$\approx 2\%$
Scăderea medie a emoțiilor negative	primele vs. ultimele 2 săpt.	$\approx 6\%$
Relevanța răspunsurilor (complet pe subiect)	subset etichetat	$\approx 72\%$
Utilizabilitate generală	rating mediu (1–5)	$\approx 3,8$

Acuratețea de circa 63% la clasificarea emoțiilor poate părea modestă, dar trebuie citită în raport cu dificultatea sarcinii. Eticheta emoțională a unui mesaj este subiectivă: același text poate fi perceput de un cititor ca tristete, iar de altul ca neutralitate. Acest lucru se vede chiar în acordul dintre cei trei adnotatori umani, de doar 79%; cu alte cuvinte, nici oamenii nu cad complet de acord, iar acest 79% reprezintă un plafon realist pentru orice clasificator automat, nu 100%. Raportat la el, modelul atinge aproximativ patru cincimi din performanța umană. În plus, cele mai multe erori nu au fost grave, ci ezitări între etichete învecinate (de exemplu „trist” și „neutru”), nu confuzii între emoții opuse, ceea ce face precizia suficientă pentru un grafic orientativ.

6.3 Evoluția stării emoționale

Cel mai relevant indicator legat de starea utilizatorului provine chiar din datele aplicației. Comparând media emoțiilor negative din primele două săptămâni cu

cea din ultimele două, am observat o scădere la patru din cei cinci utilizatori, în medie de aproximativ șase procente (de la circa 58% la circa 52%), așa cum arată Figura 6.1. Reducerea nu a fost uniformă: la un utilizator a fost clară (16 procente), la altul practic absentă. Această tendință este încurajatoare, dar trebuie citită cu prudență, deoarece este o corelație, nu o dovadă de cauzalitate: scăderea poate ține și de revenirea naturală spre medie sau de evenimente din viața utilizatorilor, nu neapărat de aplicație.

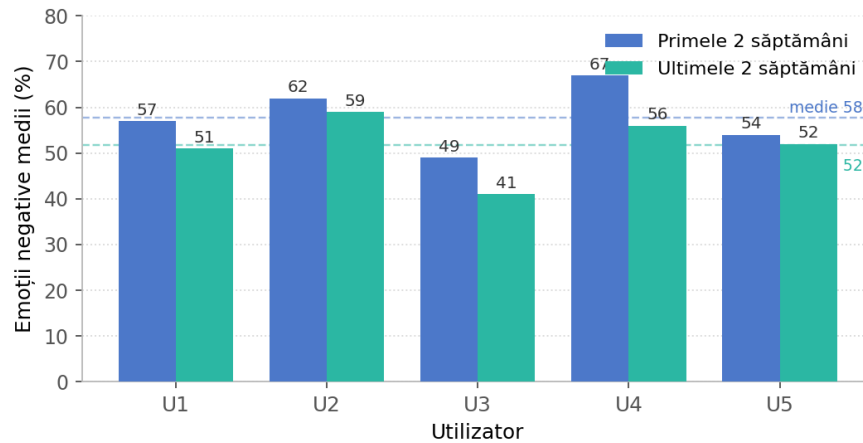


Figura 6.1: Media emoțiilor negative (tristețe + anxietate) per utilizator, la începutul și la finalul perioadei de testare

6.4 Calitatea răspunsurilor și utilizabilitatea

Răspunsurile generate prin API-ul DeepSeek au fost în ansamblu bune. Pe mesajele simple, răspunsul a fost aproape întotdeauna pe subiect; pe cele cu sens mai subtil, aproximativ trei din patru au rămas complet relevante, restul deviind spre încurajări generice, modelul a părut uneori mai preocupat să ofere confort decât să răspundă punctual. Pe partea de utilizare, participanții au considerat aplicația ușoară și intuitivă (rating mediu în jur de 4 din 5) și nu am primit întrebări de orientare, semn de navigare clară. Singura nemulțumire constantă a fost lipsa unei memorii pe termen lung: aplicația reține contextul unei conversații, dar nu detalii de la o zi la alta, ceea ce a redus pentru unii senzația de relație personală.

6.5 Amenințări la validitate

Rezultatele trebuie citite cu prudență. Eșantionul este mic, doar cinci utilizatori, aleși prin conveniență dintre persoane apropiate de vârsta și mediul meu, ceea ce limitează generalizarea. Indicatorii tehnici (acuratețe, recall, latență) sunt obiectivi și reproductibili, însă scăderea emoțiilor negative este un semnal corelativ, nu

o măsură de eficacitate: fără un grup de control și fără un instrument psihometric validat aplicat înainte și după, nu pot atribui această evoluție aplicației. De asemenea, aprecierea relevanței răspunsurilor s-a bazat parțial pe observația mea. Privite în acest context, cifrele din Tabelul 6.1 sunt indicative și încurajatoare, dar nu constituie o dovadă a eficacității.

Capitolul 7

Concluzii și Direcții Viitoare

Lucrarea de față a urmărit proiectarea și implementarea unui companion digital pentru sănătatea mintală, care să combine accesibilitatea unei aplicații mobile cu profunzimea unui chatbot bazat pe un model mare de limbaj. La nivel de implementare, obiectivele tehnice propuse în Capitolul 1 au fost îndeplinite: aplicația dispune de autentificare cu verificare prin email, conversații cu context persistent, mod privat fără stocarea conversațiilor, vizualizare grafică a evoluției emoționale, sistem de reminders locale, blocare prin PIN cu deblocare biometrică opțională și un mecanism de safety check. Evaluarea cu utilizatori reali din Capitolul 6, desfășurată pe parcursul a aproximativ trei luni și a circa 1.900 de mesaje, arată că aceste funcționalități nu doar există, ci funcționează rezonabil în uz cotidian, fără însă a demonstra o eficacitate clinică, ceea ce ar depăși cadrul acestei lucrări.

Testarea a confirmat punctele forte, dar a scos la iveală și limite concrete. Răspunsurile conversaționale au fost în general pe subiect, dar tendința modelului de a oferi confort generic în locul unui răspuns punctual rămâne vizibilă la mesajele mai subtile. Clasificarea emoțiilor, corectă în aproximativ două treimi din cazuri față de un set etichetat de trei adnotatori, este suficientă pentru un grafic orientativ, dar nu pentru decizii cu miză. Iar pe partea de safety (siguranță), pragurile alese (patru mențiuni suicidale în șapte zile, media de tristețe peste 50%) sunt empirice, stabilite din raționament, nu calibrate pe date reale. Un mecanism care decide când blochează accesul unei persoane vulnerabile ar trebui validat mult mai riguros.

Cea mai clară concluzie venită din partea utilizatorilor a fost nevoia de personalizare: aplicația reține contextul unei conversații, dar uită detaliile de la o zi la alta, ceea ce reduce senzația de relație reală. Aceasta este, de altfel, și principala direcție de dezvoltare. Adăugarea unei memorii pe termen lung, un profil care să rețină informații relevante despre utilizator și să le aducă în context la conversațiile următoare, ar răspunde direct feedbackului primit. Dincolo de aceasta, o evaluare mai amplă, cu mai mulți utilizatori și de preferință în colaborare cu specialiști, ar permite calibrarea pragurilor de detecție și verificarea reală a utilității mesajelor de sprijin. Pași complementari ar fi mutarea backend-ului pe un server propriu cu certificat TLS (de exemplu, DigitalOcean sau Railway), un jurnal personal care să

combine starea emoțională cu date despre somn și activități, și înlocuirea validării prin contact de încredere cu o validare profesională.

În concluzie, MindBuddy nu este și nu își propune să devină un înlocuitor al unui specialist. În forma actuală este un prototip funcțional, apreciat de primii utilizatori și cu limite asumate deschise, un punct de plecare credibil, nu o soluție validată: o încercare de a construi un instrument digital care ascultă, organizează emoțiile și oferă un cadru de auto-reflecție, un pas mic, dar concret, către o abordare mai accesibilă a sănătății mintale.

Bibliografie

- [1] World Health Organization, *World Mental Health Report: Transforming Mental Health for All*. Geneva: World Health Organization, 2022.
- [2] R. C. Kessler, P. Berglund, O. Demler, R. Jin, K. R. Merikangas, and E. E. Walters, “Lifetime prevalence and age-of-onset distributions of DSM-IV disorders in the national comorbidity survey replication,” *Archives of General Psychiatry*, vol. 62, no. 6, pp. 593–602, 2005.
- [3] K. K. Fitzpatrick, A. Darcy, and M. Vierhile, “Delivering cognitive behavior therapy to young adults with symptoms of depression and anxiety using a fully automated conversational agent (woebot): A randomized controlled trial,” *JMIR Mental Health*, vol. 4, no. 2, p. e19, 2017.
- [4] D. D. Luxton, *Artificial Intelligence in Behavioral and Mental Health Care*. London: Academic Press, 2016.
- [5] B. Pang and L. Lee, “Opinion mining and sentiment analysis,” *Foundations and Trends in Information Retrieval*, vol. 2, no. 1–2, pp. 1–135, 2008.
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [7] DeepSeek-AI, “DeepSeek LLM: Scaling open-source language models with longtermism,” *arXiv preprint arXiv:2401.02954*, 2024.
- [8] J. Torous, J. Nicholas, M. E. Larsen, J. Firth, and H. Christensen, “Clinical review of user engagement with mental health smartphone apps: evidence, theory and improvements,” *Evidence-Based Mental Health*, vol. 21, no. 3, pp. 116–119, 2018.
- [9] ngrok Inc., “ngrok — unified application delivery platform,” 2024. Accesat: martie 2026.
- [10] Google LLC, “Jetpack compose — Android’s modern toolkit for building UI,” 2023. Accesat: martie 2026.
- [11] OpenJS Foundation, “Express.js — fast, unopinionated, minimalist web framework for Node.js,” 2024. Accesat: martie 2026.

- [12] PostgreSQL Global Development Group, “PostgreSQL 16 documentation,” 2024. Accesat: martie 2026.
- [13] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proceedings of the USENIX Annual Technical Conference*, pp. 32–32, 1999.
- [14] M. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” RFC 7519, Internet Engineering Task Force, 2015.
- [15] Google, “Workmanager — android developers,” 2026. Accesat: mai 2026.
- [16] K. Kroenke, R. L. Spitzer, and J. B. W. Williams, “The patient health questionnaire-2: Validity of a two-item depression screener,” 2003. Accesat: martie 2026.
- [17] Square Inc., “Retrofit — a type-safe HTTP client for Android and Java,” 2024. Accesat: martie 2026.
- [18] OWASP Foundation, “OWASP mobile application security verification standard (MASVS),” 2023. Accesat: martie 2026.

Anexa A

Anexă - Utilizarea instrumentelor de inteligență artificială generativă

În această anexă este prezentată o sinteză a modului în care instrumentele de inteligență artificială generativă au fost utilizate în cadrul lucrării. O imagine de ansamblu este oferită în Tabelul A.1, iar detalierea pe fiecare instrument în secțiunile următoare.

Instrument AI	Scopul utilizării	Secțiuni în care a fost utilizat	Observații
DeepSeek API	Generarea răspunsurilor conversaționale empatic	Cap. 3 (3.2.4), Cap. 4 (4.2)	Componentă integrată în aplicația finală, accesată prin API
	Clasificarea emoțiilor din mesajele utilizatorului	Cap. 4 (4.4), Cap. 5 (5.4)	Răspuns în format JSON, validat înainte de stocare
Claude (Anthropic)	Depanarea unor erori nefamiliare și clarificarea unor abordări standard de implementare	Cap. 4(4.2.1, 4.5, 4.6), Cap. 5 (5.6, 5.7.3)	Asistent în dezvoltare; nu apare în aplicația finală; cod adaptat și retestat

Tabelul A.1: Sinteza utilizării instrumentelor de inteligență artificială generativă

A.1 Detalii privind utilizarea DeepSeek

DeepSeek API este singurul instrument AI integrat efectiv în produsul final, accesat prin API în două roluri. Pentru generarea răspunsurilor, modelul `deepseek-chat` (temperatura 0.7) primește un mesaj de sistem care îl obligă să răspundă empatic și exclusiv în limba română, împreună cu istoricul conversației reasamblat la fiecare

cerere (Secțiunile 3.2.4 și 4.2). Pentru clasificarea emoțiilor, același model (temperatura 0, pentru determinism) primește un prompt strict care cere un obiect JSON cu distribuția procentuală a cinci emoții (Secțiunile 4.4 și 5.4).

Rezultatele nu sunt preluate integral: răspunsul de clasificare este validat înainte de a fi acceptat (chei prezente, valori întregi între 0 și 100, sumă exact 100); dacă validarea eșuează, datele emoționale nu sunt stocate. Promptul de sistem și parametrii au fost stabiliți și ajustați de mine prin testare repetată, iar integrarea s-a făcut integral în backend-ul propriu.

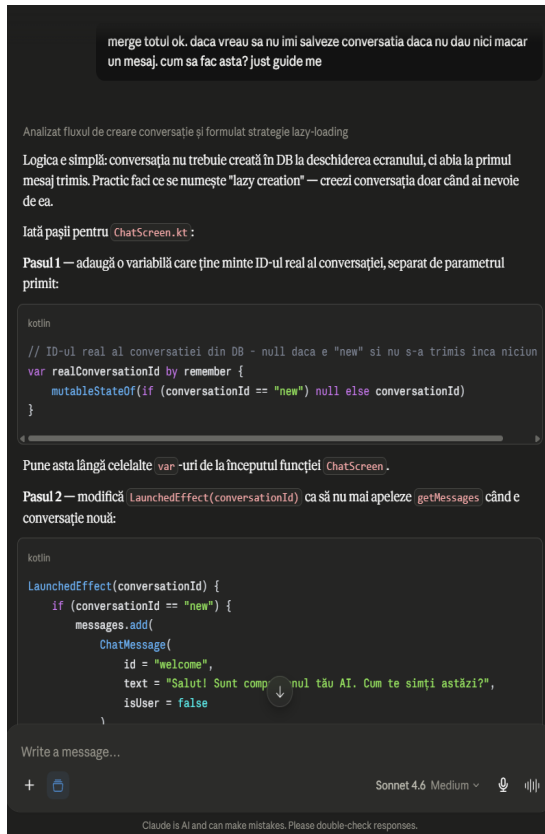
A.2 Detalii privind utilizarea Claude

Claude (Anthropic) a fost folosit exclusiv ca asistent în procesul de dezvoltare și nu apare în aplicația finală sub nicio formă. L-am consultat strict atunci când întâmpinam o eroare al cărei mesaj nu îmi era familiar (de exemplu, integrarea bibliotecii `BiometricPrompt`, conflicte de stare în Jetpack Compose între ecrane care se închid și se redeschid, sau erori de drepturi în PostgreSQL la operații pe tabele noi) sau atunci când nu cunoșteam o abordare standard pentru o problemă concretă, cum este reprogramarea unui `PeriodicWorkRequest` după restart-ul dispozitivului.

Unele erori au fost greu de izolat doar din mesajul afișat în consolă și s-au rezolvat abia după ce am conectat Claude direct la proiect, cu acces la codul sursă; cu contextul real al fișierelor, sugestiile au devenit punctuale, nu generice (Figura A.1).

Niciun fragment generat nu a fost preluat ca atare. Codul oferit a fost adaptat la convențiile proiectului: nume de variabile consistente cu restul aplicației, comentarii rescrise în stilul personal pe care îl folosesc (concise, în limba română, fără explicații redundante) și logică integrată în arhitectura existentă, nu adăugată ca un bloc separat. Pentru fluxurile mai complexe, cum este deblocarea stării de criză prin testul PHQ-2 combinat cu validarea unui contact de încredere, am ales varianta finală abia după ce am cântărit alternativele. Fiecare soluție a fost verificată și retestată practic înainte de a fi păstrată.

Îmi asum integral conținutul: ideile din text îmi aparțin, iar codul aplicației a fost scris în proporție de circa 60% de mine de la zero, restul pornind de la idei verificate și integrate manual. Instrumentele de inteligență artificială au fost folosite ca ceea ce sunt, instrumente, fără a le delega rolul de autor.



(a) Crearea lazy a conversației



(b) Eroare de drepturi în PostgreSQL

Figura A.1: Două exemple de asistență oferită de Claude în etapa de depanare: ghidarea pas cu pas pentru crearea amânată a conversației (a) și rezolvarea erorii `permission denied for table` la operațiile pe tabele noi (b).